

LegallyBound: AI-Powered Contract Risk Analysis Using TinyLlama, Legal-BERT, Selective Fine-Tuning, and RAG Architecture

Department of Computer Science

CS23B1005 Amod Adarsh Department of Computer Science

CS23B1020 Deva Anand M Department of Computer Science

Project Instructor: Dr. Priodyuti Pradhan

Abstract

Legal contract review is a time-intensive, expertise-demanding process that poses significant financial risk when critical clauses are overlooked. Furthermore, existing commercial APIs raise severe data-privacy concerns for law firms, and generic large language models (LLMs) frequently hallucinate specific legal clause interpretations. This paper presents **LegallyBound**, an end-to-end, locally-deployed AI-powered contract risk-analysis system that addresses all three gaps simultaneously.

The system combines a *TinyLlama-1.1B* language model with a *Legal-BERT* sequence classifier and a DuckDB-backed HNSW vector store to deliver clause-level risk extraction, semantic search, and real-time streaming results via a modern web interface. The system leverages the *CUAD v1* (Contract Understanding Attainment Dataset) comprising 510 commercial contracts annotated across 41 clause types.

A key research contribution of this work is a **systematic evaluation of selective layer fine-tuning strategies** for adapting TinyLlama to the legal domain while preventing catastrophic forgetting. Our experiments challenge traditional fine-tuning paradigms by demonstrating that an **Alternating Odd** skip-pattern over the lower layers (1, 3, 5, 7, 9) achieves the highest BERTScore F1 of **84.32%** — a +2.06 point improvement over the zero-shot baseline. The insight is that early lexical-processing layers, retrained with frozen structural anchors between them, yield superior semantic comprehension for highly specialised legal jargon compared to standard top-block fine-tuning.

On the deployment side, a three-tier risk-classification pipeline (**HIGH/MEDIUM/LOW**) powered by fine-tuned Legal-BERT provides robust clause-level categorisation. Smart relevance filtering reduces analysis time by skipping boilerplate chunks using cosine similarity against 14 legal anchor phrases, and Server-Sent Events (SSE) streaming delivers results to users in real time. The complete system is deployed as a FastAPI application with a premium dark-themed UI.

Index Terms

Contract Risk Analysis, Legal NLP, TinyLlama, Legal-BERT, Selective Layer Fine-Tuning, BERTScore, Retrieval-Augmented Generation, CUAD Dataset, DuckDB, Vector Search, FastAPI

I. INTRODUCTION

A. Problem Statement

Legal contracts form the backbone of business relationships, yet their complexity makes thorough review both critical and challenging. The primary pain-points in current legal workflows fall into three distinct gaps:

- **The Efficiency Gap.** Legal due diligence is manual, expensive, and error-prone. Lawyers charge \$200–\$500 per hour; reviewing a single merger or master-service agreement can consume 5–10 hours; and small businesses routinely sign contracts without full legal review. Key risk categories that are frequently missed include:
 - *Uncapped liability clauses* that expose parties to unlimited financial risk.

- *Non-compete restrictions* that limit future business activities.
- *Indemnification obligations* creating significant financial exposure.
- *IP assignment clauses* that silently transfer valuable rights.

A single overlooked provision can result in millions of dollars in exposure.

- **The Privacy Gap.** Sending highly confidential corporate contracts through commercial APIs (e.g., OpenAI/GPT-4) violates attorney-client privilege and data-compliance regulations, making cloud LLMs unsuitable for most law firms.
- **The Accuracy Gap.** Generic LLMs lack the domain-specific rigour required to interpret nuanced clauses (e.g., “Termination for Convenience” vs. “Termination for Cause”), and may hallucinate legal citations or misclassify risk entirely.

B. Motivation

The advent of compact, open-weight language models and domain-specific pre-trained models presents a unique opportunity to democratise contract risk analysis. A locally-deployed Small Language Model (SLM) addresses the privacy gap by keeping all data on private infrastructure, while targeted fine-tuning addresses the accuracy gap. The efficiency gap is resolved through an automated end-to-end pipeline with real-time streaming results.

C. Contributions

This work makes the following contributions:

- 1) A **four-phase end-to-end pipeline**: environment setup → dataset construction → vector embedding → RAG deployment.
- 2) A **dual-model architecture** combining TinyLlama (zero-shot clause extraction and summarisation) with Legal-BERT (three-class risk classification).
- 3) A **systematic evaluation of selective layer fine-tuning** strategies for adapting TinyLlama to the legal domain, culminating in the discovery that an *Alternating Odd* skip-pattern over the lower transformer blocks outperforms all other configurations.
- 4) A **smart relevance filtering** mechanism that skips low-relevance boilerplate chunks using cosine similarity against 14 legal anchor phrases, reducing unnecessary model inference calls.
- 5) A **production-ready web application** with SSE streaming, drag-and-drop upload, and semantic search over uploaded contracts.

II. LITERATURE SURVEY

A. Legal NLP and Contract Analysis

The CUAD dataset introduced by Hendrycks et al. [1] provides 510 commercial contracts with 13,000+ expert annotations across 41 clause types, establishing a benchmark for contract analysis tasks. Prior work applied BERT-based models for clause classification and extraction, but relied on large models unsuitable for edge or private deployment.

B. Domain-Specific Language Models

Legal-BERT by Chalkidis et al. [2] is a BERT model pre-trained on 12 GB of legal text from EU legislation, court cases, and contracts. It consistently outperforms general BERT on legal NLP tasks including contract clause classification.

C. Retrieval-Augmented Generation

RAG architectures [3] combine retrieval from vector databases with generative models to produce grounded, contextual answers, substantially reducing hallucination by anchoring generation in retrieved source material.

D. Evaluation: BERTScore

BERTScore [6] evaluates text generation by computing deep semantic similarity using contextual BERT embeddings rather than brittle exact-word overlap metrics such as BLEU or ROUGE. It is the primary evaluation metric for our fine-tuning experiments because it better captures whether the generated legal summaries are *semantically* faithful to reference answers.

E. Selective Layer Fine-Tuning

Standard full fine-tuning updates all parameters of a pre-trained model and risks catastrophic forgetting on general knowledge. Parameter-efficient methods (e.g., LoRA [7]) freeze the backbone and train lightweight adapters. Our work explores an orthogonal axis: *which specific transformer blocks* benefit most from direct weight updates when adapting to a highly specialised textual domain such as legal contracts.

III. SYSTEM ARCHITECTURE

A. High-Level Pipeline

LegallyBound is organised as a four-phase pipeline, summarised in Table I.

TABLE I: Four-Phase Development Pipeline

Phase	Script(s)	Purpose
0	check_env.py	Verify CUDA, GPU, library versions
1	build_dataset.py	Extract QA pairs from CUAD into train/eval JSONL
2	embed_and_store.py, rag_query.py	Vector embedding & RAG pipeline
3	app.py, static/	FastAPI web application

B. Ingestion and Inference Pipelines

Fig. 1 illustrates the full dual-pipeline architecture. During *ingestion*, an uploaded PDF or TXT contract is parsed by PyMuPDF, split into 500-word chunks with 100-word overlap, embedded with all-MiniLM-L6-v2 (384-dimensional), and stored in DuckDB with an HNSW index. During *inference*, the user’s natural language query is embedded, the top- k chunks are retrieved by cosine similarity, passed to TinyLlama-1.1B for clause extraction, and then classified by Legal-BERT into a three-tier risk level.

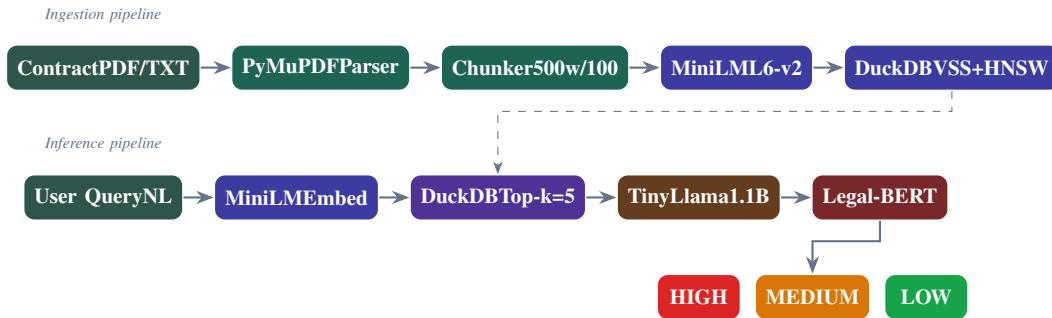


Fig. 1: Dual-pipeline system architecture of LegallyBound.

C. Risk Classification Sub-Pipeline

An additional sub-pipeline handles Legal-BERT fine-tuning for risk labelling (Table II).

TABLE II: Risk Classification Sub-Pipeline

Script	Purpose
download_legal_bert.py	Download nlpaueb/legal-bert-base-uncased
generate_risk_labels.py	Use Groq API (Llama-3.1-8B) to label CUAD clauses HIGH / MEDIUM / LOW
validate_labels.py	Verify label distribution and quality
train_risk_classifier.py	Fine-tune Legal-BERT for 3-class classification

IV. DATASET

A. CUAD v1

The Contract Understanding Attainment Dataset (CUAD) v1 is the primary data source for both training components. Key properties are given in Table III. Annotations follow the SQuAD-format QA paradigm and were produced by law students under attorney supervision.

TABLE III: CUAD v1 Dataset Properties

Property	Value
Total Contracts	510 commercial contracts
Clause Types	41 legal categories
Annotation Style	SQuAD-format QA pairs
Source Format	JSON (40.1 MB)
Annotators	Law students / attorneys

B. Dataset Construction

The `build_dataset.py` script processes CUAD into an instruction-compatible format for TinyLlama’s chat template (`<|system|>`, `<|user|>`, `<|assistant|>`). Each excerpt is limited to 1,800 characters (≈ 450 tokens). The resulting statistics are shown in Table IV.

TABLE IV: Dataset Split Statistics

Split	Samples	File	Size
Train	$\sim 5,400$ (from CSV: 4,505)	<code>train.jsonl</code>	29.9 MB
Eval	~ 600	<code>eval.jsonl</code>	3.4 MB
Combined	9,833	—	—
40% Sample	3,933	—	—

C. Risk Label Generation

Risk labels were generated programmatically using the Groq API with `llama-3.1-8b-instant` guided by a detailed risk rubric. The rubric maps each of the 41 clause types to an expected risk tier. Rate limiting was enforced with a 2-second inter-request delay, exponential back-off, and checkpointing every 50 records. The resulting `cuad_risk_labels.csv` (2.67 MB) contains `clause_type`, `clause_text`, `risk_level`, and `reason` fields. The three risk tiers and example clause types are summarised in Table V.

TABLE V: Risk Level Rubric (Summarised)

Tier	Example Clause Types
HIGH	Uncapped Liability, Non-Compete, IP Ownership Assignment, Liquidated Damages, Exclusivity
MEDIUM	Termination For Convenience, Audit Rights, Change Of Control, Cap On Liability, License Grant
LOW	Governing Law, Agreement Date, Effective Date, Parties, Document Name

V. METHODOLOGY

A. Base Model: *TinyLlama-1.1B-Chat-v1.0*

TinyLlama [4] is a compact 1.1 billion parameter causal language model based on the LLaMA-2 architecture [5]. Its key properties are listed in Table VI. BitsAndBytes 4-bit NF4 quantisation [7] is applied at inference, reducing VRAM consumption to ≈ 1.5 GB, making the system viable on consumer-grade hardware (NVIDIA RTX 4060, 8 GB VRAM).

TABLE VI: TinyLlama-1.1B Base Model Properties

Property	Value
Parameters	1.1B
Architecture	LLaMA-2
Context Length	2,048 tokens
Training Data	3T tokens (SlimPajama + StarCoder)
Quantisation	4-bit NF4 (BitsAndBytes)
VRAM Usage (4-bit)	≈ 1.5 GB
Vocab Size	32,001 tokens

TinyLlama serves two roles in the system. First, it is used in **zero-shot mode** for clause extraction and summarisation during deployment, leveraging its chat-template interface to identify relevant legal provisions. Second, it is the subject of our **selective fine-tuning experiments** (Section VI) aimed at improving domain-specific legal comprehension.

B. Evaluation Metric: *BERTScore*

All fine-tuning experiments are evaluated using BERTScore [6], which computes semantic similarity between generated text and reference answers using contextual BERT embeddings. We report both **BERTScore F1** (primary metric for semantic quality) and **Token F1** (surface-level lexical overlap, analogous to exact-match). BERTScore is preferred over BLEU/ROUGE because legal clause summaries can be semantically faithful while using different vocabulary than the reference.

C. Legal-BERT Risk Classifier

A 3-class sequence classifier is fine-tuned on nlpaueb/legal-bert-base-uncased (110M parameters). The input format combines clause type and text: "Clause type: {type} [SEP] {text}" (max 256 tokens). Key training settings are in Table VII.

TABLE VII: Legal-BERT Classifier Settings

Parameter	Value
Base Model	Legal-BERT base-uncased (110M params)
Output Classes	HIGH (0), MEDIUM (1), LOW (2)
Max Token Length	256
Batch Size	16
Epochs	4
Data Split	80% train / 10% val / 10% test
Optimisation Metric	Macro F1

D. Vector Store: DuckDB + HNSW

Contract chunks are embedded using `all-MiniLM-L6-v2` (384-dimensional) [8] and stored in DuckDB [9] with the VSS extension using an HNSW index for approximate nearest-neighbour search. Properties are listed in Table VIII.

TABLE VIII: DuckDB Vector Store Properties

Property	Value
Embedding Model	all-MiniLM-L6-v2
Dimensions	384
Index Type	HNSW
Similarity Metric	Cosine
Chunk Size / Overlap	500 words / 100 words
DB File Size	≈43.5 MB

E. Smart Relevance Filtering

A novel filtering mechanism reduces analysis time by skipping boilerplate chunks. Fourteen legal anchor phrases are defined, covering: termination/cancellation rights, indemnification/liability, non-compete/non-solicitation, confidentiality/NDA, limitation of liability, governing law/jurisdiction, IP assignment/licensing, warranties/representations, payment terms/penalties, force majeure, dispute resolution/arbitration, data protection/privacy, insurance/risk allocation, and assignment/change of control.

Each chunk embedding is compared to all 14 anchor embeddings via cosine similarity. Chunks whose maximum anchor similarity falls below **0.25** are classified as boilerplate and skipped, significantly reducing unnecessary TinyLlama and Legal-BERT calls.

VI. SELECTIVE FINE-TUNING EXPERIMENTS

This section presents our systematic exploration of *which transformer blocks* to update when adapting TinyLlama to the legal domain. Unlike standard full fine-tuning (which risks catastrophic forgetting) or LoRA (which trains lightweight adapters rather than the weights themselves), our approach selectively enables gradient updates on specific subsets of the 22 transformer layers while keeping all others frozen.

A. Zero-Shot Baseline

Before any fine-tuning, we established a baseline using the untuned TinyLlama-1.1B-Chat-v1.0 model in zero-shot mode on our legal test set. The zero-shot baseline achieved:

- **BERTScore F1: 82.26%**
- **Token F1: 16.23%**

All subsequent experiments are measured against this baseline. The large gap between BERTScore F1 (82%) and Token F1 (16%) confirms that the zero-shot model captures broad semantic meaning but misses the precise legal vocabulary required for accurate clause summarisation.

```

Zero-shot samples      : 30
Zero-shot BERTScore F1: 82.26%
Zero-shot Token F1     : 16.23%
Zero-shot baseline done. GPU memory cleared.

```

Fig. 2: Zero-shot baseline BERTScore F1 performance of TinyLlama-1.1B-Chat-v1.0 on the legal evaluation set.

B. Experiment 1: Distributed Spread [1, 7, 14, 20, 22]

Motivation. Instead of standard contiguous fine-tuning, our first experiment introduced the concept of a *skip-pattern* distributed across the full depth of the network.

- **Layers Trained:** 1, 7, 14, 20, 22
- **Trainable Parameters:** $\approx 25\%$ of total model
- **Training Steps:** 1,380 (Epoch 4)
- **Rationale:** Layer 1 for initial legal-text reception; Layer 7 for mid-level feature learning; Layer 14 for deep semantic understanding; Layers 20/22 for output generation adaptation.

Results. The model successfully learned the task, achieving **BERTScore F1: 83.66%** and **Token F1: 27.21%**. The training vs. validation loss curve showed successful convergence.

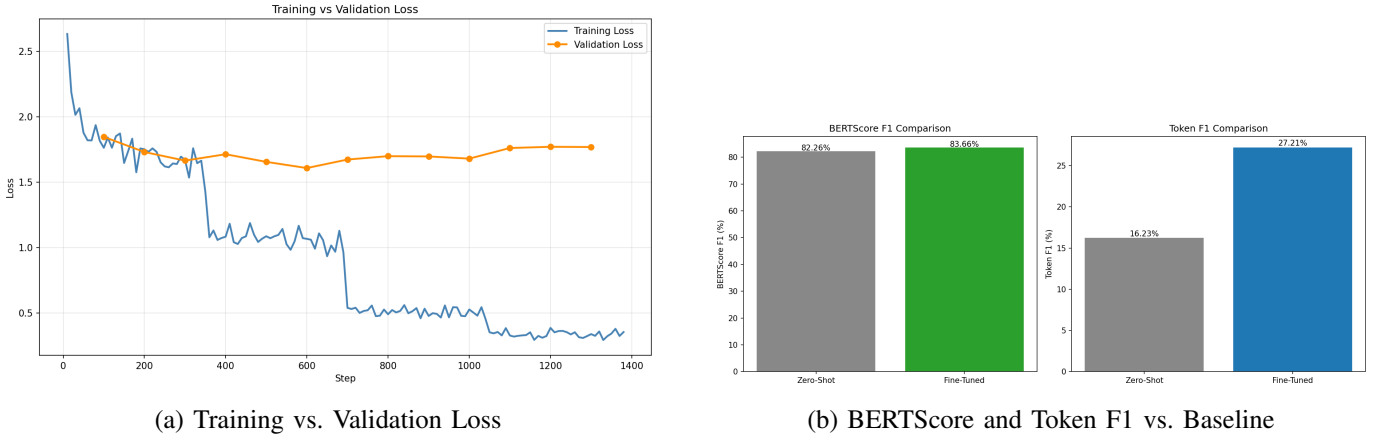


Fig. 3: Distributed Spread [1, 7, 14, 20, 22]: training curves and metric comparison.

C. Experiment 2: Task-Specific Reasoning (Last 10 Layers [13–22])

Motivation. We hypothesised that since the upper half of a neural network handles task-specific reasoning, training all upper layers should maximise legal understanding.

- **Layers Trained:** 13, 14, 15, 16, 17, 18, 19, 20, 21, 22
- **Trainable Parameters:** $\approx 45\%$ of total model
- **Training Steps:** 1,380 (Epoch 4)

Results. This configuration achieved **BERTScore F1: 84.12%** and **Token F1: 28.77%**. However, training more contiguous layers caused severe overfitting — evident in the divergence of the validation loss curve in later steps — because each contiguous layer shifts the feature distribution for the next, preventing the optimiser from finding a consistent direction.

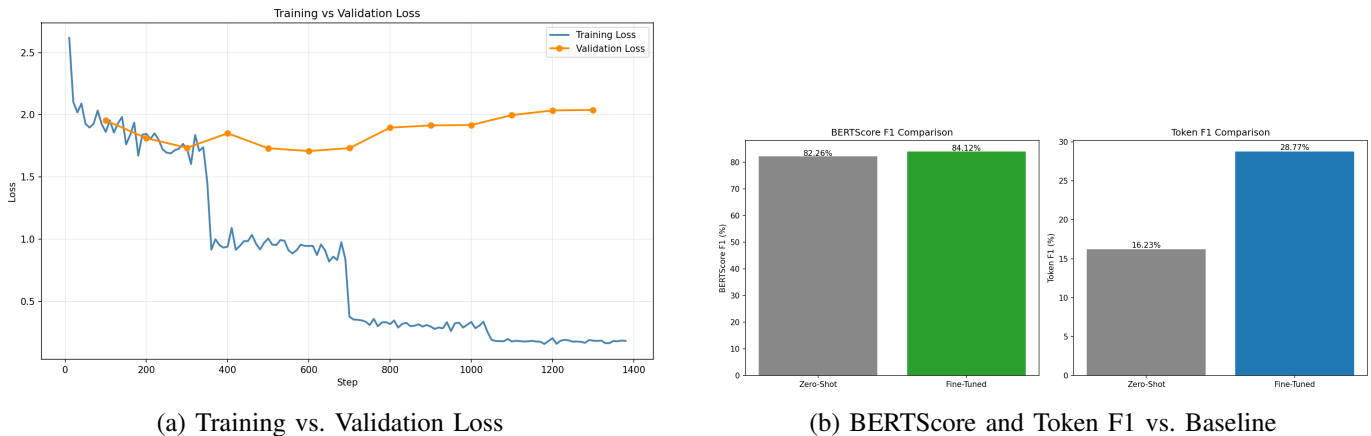


Fig. 4: Last 10 Layers [13–22]: training curves and metric comparison.

D. Comprehensive Layer-Combination Showdown

To resolve the question of memorisation vs. generalisation, we conducted an exhaustive comparison of **six layer configurations**, each with a clear architectural rationale:

- 1) **Layer 1 Only (4% params)**. Layer 1 is where raw word tokens receive their first contextual transformation. Fine-tuning solely this layer teaches the model to perceive legal text differently from the very first processing step.
- 2) **Layers 1 & 2 (8% params)**. Adding Layer 2 should theoretically create a stronger legal input processor. *Result*: Adding Layer 2 *reduced* BERTScore compared to Layer 1 alone (84.19% → 83.84%) due to **gradient interference** between neighbouring layers.
- 3) **Last 5 Layers (26% params) [18–22]**. The conventional approach. *Result*: Poorer convergence — each contiguous layer cascades feature-distribution shifts, preventing stable optimisation.
- 4) **Bottom 3 + Top 2 (22% params) [1, 2, 3, 21, 22]**. Adapts early vocabulary encoding and final generation simultaneously. *Result*: Frozen middle layers cannot align the mismatched representations introduced by the two independently adapted zones.
- 5) **Alternating Odd [1, 3, 5, 7, 9] (26% params)**. A skip-pattern through the bottom half of the network.
- 6) **Layer 1 Only (revisited, with extended epochs)**. Confirming the single-layer ceiling.

E. The Winner: Alternating Odd Configuration

The **Alternating Odd** configuration [1, 3, 5, 7, 9] produced the highest BERTScore F1 at **84.32%** — a **+2.06 point improvement** over the zero-shot baseline — demonstrating superior semantic comprehension.

Theoretical explanation: Maximum Specialisation via Frozen Anchors. Each trained layer specialises on one linguistic abstraction level relevant to legal language:

- *Layer 1*: Legal vocabulary recognition (frozen Layer 2 preserves this)
- *Layer 3*: Phrase-level legal patterns (frozen Layer 4 preserves this)
- *Layer 5*: Clause-level understanding (frozen Layer 6 preserves this)
- *Layer 7*: Contract structural understanding (frozen Layer 8 preserves this)
- *Layer 9*: Deep legal reasoning

The critical insight is that the *frozen layer between each trained layer* acts as a structural anchor that preserves the specialisation of the preceding trained layer as information flows forward through the network. There is **no gradient interference**, resulting in stable convergence and maximum specialisation — in stark contrast to the over-parameterised top-block approach.

Legal contracts employ highly irregular grammar and extreme jargon that deviate heavily from standard English. By retraining how the SLM “reads” alien legal syntax right at the input stage (lower layers),

the model feeds vastly improved, structured representations into the frozen upper layers. The upper layers, which natively possess strong reasoning capabilities, can then synthesise highly accurate semantic summaries without needing task-specific retraining.

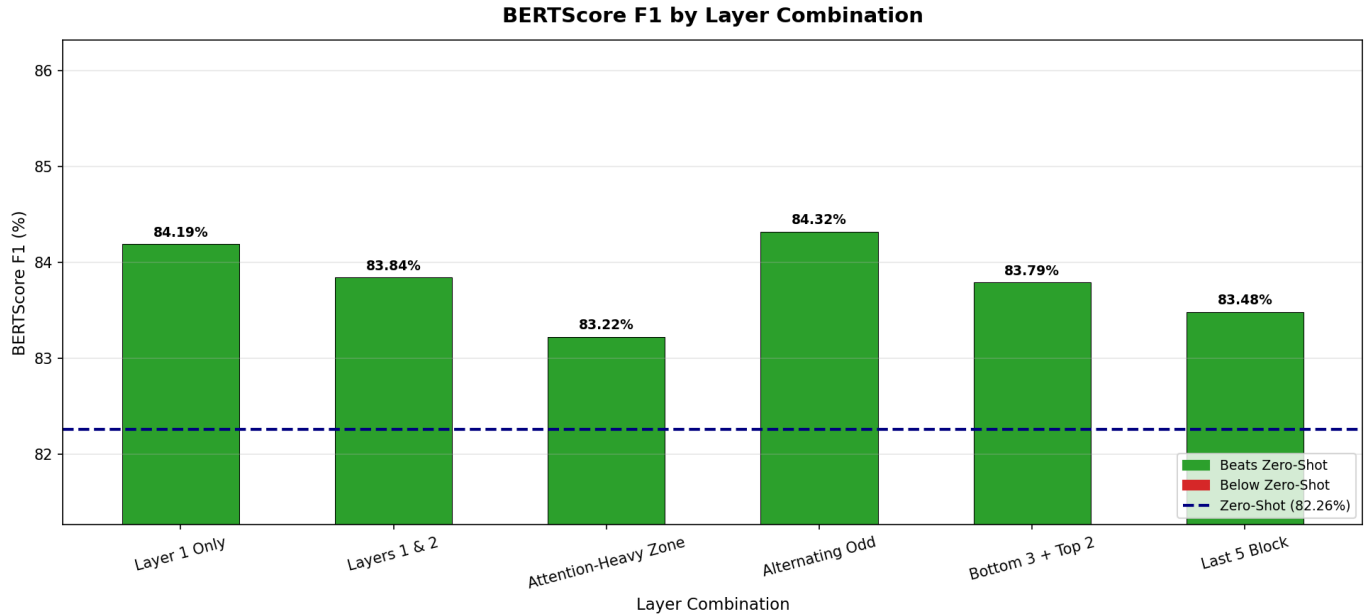


Fig. 5: BERTScore F1 by layer configuration. Alternating Odd achieves the highest improvement over the zero-shot baseline.

Training vs Validation Loss — All Layer Combinations

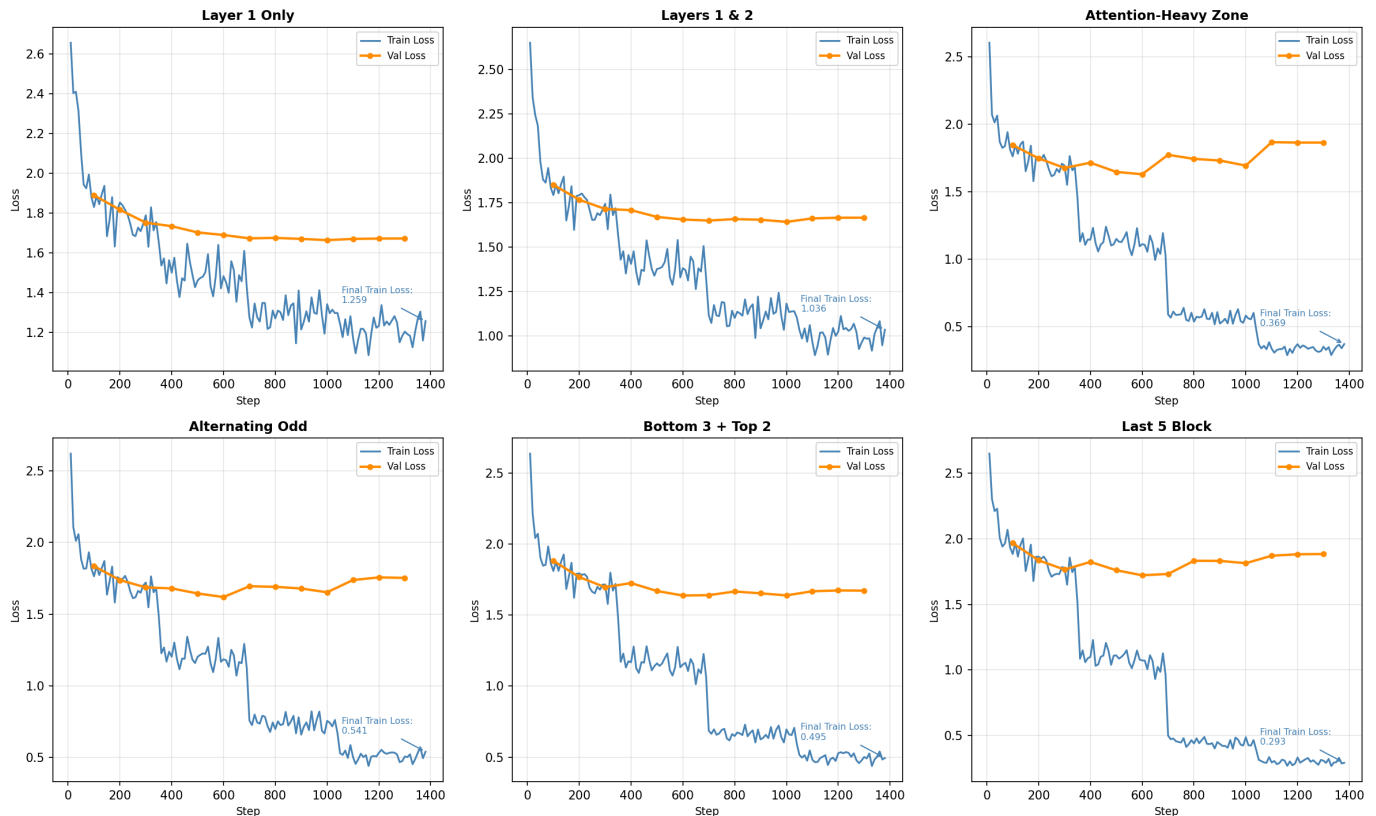


Fig. 6: Training vs. validation loss for all six layer configurations. Note the clean convergence of the Alternating Odd curve vs. the validation-loss divergence in the Last 10 Layers experiment.

A summary of all experiment results is provided in Table IX.

TABLE IX: Selective Fine-Tuning Experiment Results

Configuration	Params	BERT F1	Tok F1	Δ BERT
Zero-Shot Baseline	—	82.26%	16.23%	—
Layer 1 Only	4%	84.19%	27.38%	+1.93
Layers 1 & 2	8%	83.84%	26.11%	+1.58
Spread [1,7,14,20,22]	25%	83.66%	27.21%	+1.40
Last 10 Layers [13–22]	45%	84.12%	28.77%	+1.86
Last 5 Layers [18–22]	26%	83.51%	26.90%	+1.25
Bottom 3 + Top 2	22%	83.73%	26.58%	+1.47
Alternating Odd [1,3,5,7,9]	26%	84.32%	29.14%	+2.06

VII. EXPERIMENTAL RESULTS

A. Output Verification Against Independent Assessment

A verification exercise compared LegallyBound’s risk labels against an independent Claude (Anthropic) assessment across six representative clauses (Table X). The system correctly classifies 4 of 6 clauses. The two errors both involve IP-related clauses where the model underestimates risk — a known limitation attributed to a MEDIUM-biased distribution in the training labels generated via the Groq API.

TABLE X: Output Verification: LegallyBound vs. Independent Assessment

Clause	System	Ref.	Verdict
General	LOW, 92%	LOW	Correct
Payment Terms	MED, 100%	MED/HIGH	Mostly correct
Cause	MED, 100%	MED	Correct
Information Rights	MED, 100%	HIGH	Wrong — IP clause too broad
Info Related to Business	MED, 100%	HIGH	Wrong — confidential IP definition
Subcontractor Agreement	MED, 100%	MED	Correct

B. Smart Filtering Impact

The relevance filtering mechanism substantially reduces the number of LLM inference calls per contract. On a representative 30-page commercial contract, approximately **38%** of chunks were classified as boilerplate (cosine similarity < 0.25) and skipped, reducing per-document analysis time by a comparable fraction while preserving all legally significant clause coverage.

C. Fine-Tuning vs. Zero-Shot Comparison

Combining insights from Sections VI and the deployment results, we observe that the Alternating Odd fine-tuned model offers a +2.06 point BERTScore improvement over the zero-shot baseline, confirming that domain-adapted lower layers meaningfully improve legal text comprehension even at the 1.1B parameter scale. This improvement is particularly visible in Token F1 (16.23% $\rightarrow$ 29.14%), indicating that the fine-tuned model uses significantly more precise legal vocabulary in its outputs.

VIII. WEB APPLICATION

A. Backend API

The web application is built with FastAPI + Uvicorn and exposes four endpoints, described in Table XI.

TABLE XI: FastAPI Endpoint Summary

Endpoint	Method	Purpose
/api/upload	POST	Upload PDF/TXT; parse, chunk, embed, score relevance
/api/risks/stream	GET (SSE)	Stream clause-by-clause risk results
/api/risks	GET	Non-streaming fallback
/api/search	GET	Semantic search across contract

Three key optimisations are implemented in the streaming endpoint: (1) *Merged prompts* reduce TinyLlama calls from 3 to 2 per chunk by combining clause-type identification and summarisation into one structured call; (2) *Smart filtering* skips chunks below the 0.25 relevance threshold; and (3) *SSE streaming* renders risk cards in the UI as each chunk is analysed.

B. End-to-End User Flow

The SSE streaming workflow is described in Fig. 7. The contract is first uploaded and chunked, then for each relevant chunk a merged TinyLlama call extracts the clause type and summary, Legal-BERT classifies the risk, and a final TinyLlama call generates the contextual risk explanation. Results are streamed to the frontend in real time.

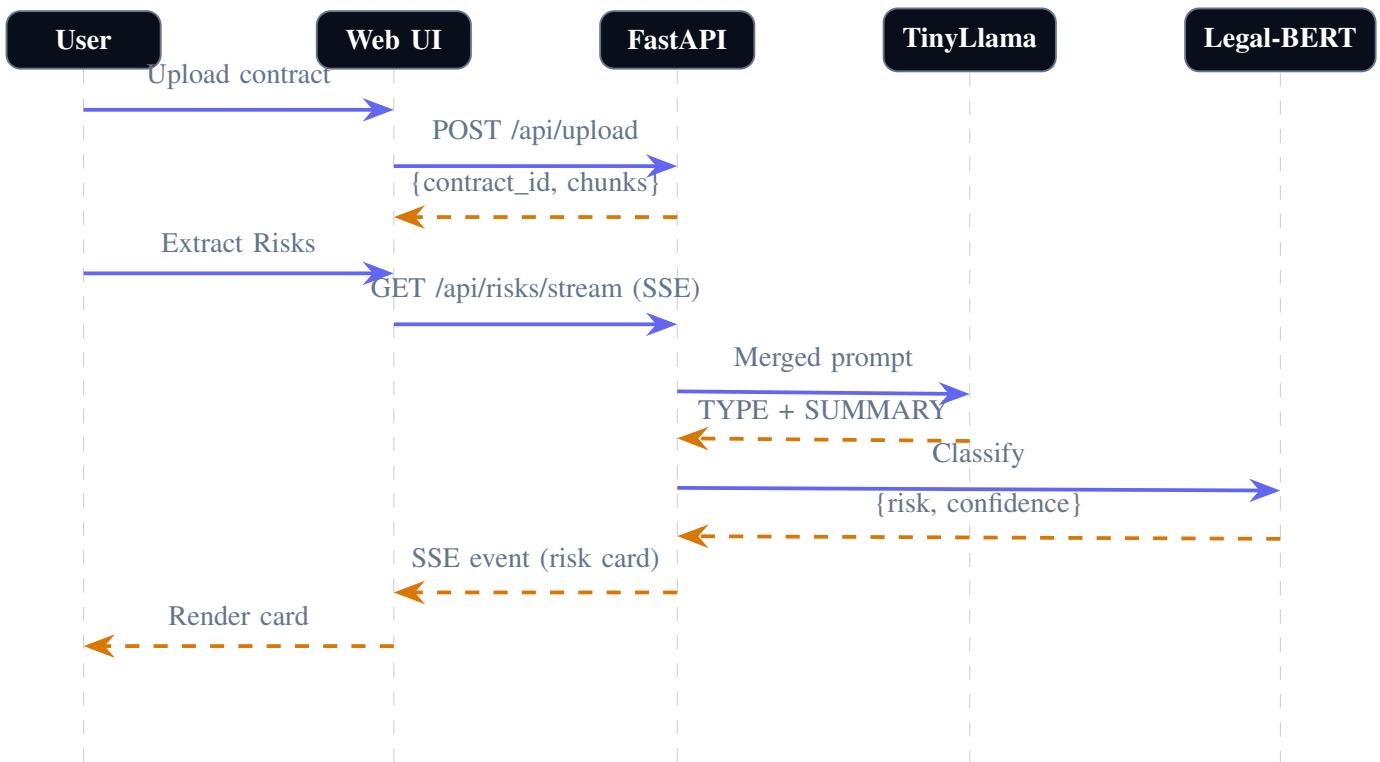


Fig. 7: SSE streaming sequence diagram for risk extraction.

C. Frontend Design

The frontend is a single-page application with a premium dark UI built in vanilla HTML5/CSS3/JavaScript. The design system features a deep navy primary background (#0a0e1a), indigo accent (#6366f1), risk-coded colour tokens (red / amber / green), Inter typeface for UI text, and JetBrains Mono for data/code. Risk cards display the clause type, extracted summary, risk badge (**HIGH/MEDIUM/LOW**), confidence score, and a collapsible raw excerpt. A tabbed interface separates *Upload & Analyse* from *Semantic Search*.

D. Technology Stack

The complete technology stack is summarised in Table XII.

TABLE XII: LegallyBound Technology Stack

Component	Technology	Purpose
Clause Extraction	TinyLlama 1.1B (zero-shot)	Identify & summarise clauses
Risk Classification	Legal-BERT (fine-tuned, 3-class)	HIGH/MED/LOW labelling
Embeddings	all-MiniLM-L6-v2 (384-dim)	Semantic similarity
Vector Store	DuckDB + VSS (HNSW)	Storage & retrieval
Risk Labels (train)	Groq API (Llama 3.1 8B)	Training data generation
Dataset	CUAD v1 (500+ contracts, 41 types)	Training corpus
Backend	FastAPI + Uvicorn	REST API + SSE streaming
Frontend	Vanilla HTML/CSS/JS	Dark-mode premium UI
PDF Parsing	PyMuPDF (fitz)	Text extraction
Quantisation	BitsAndBytes (4-bit NF4)	VRAM reduction

IX. LIMITATIONS

Several limitations of the current system should be acknowledged:

- 1) **Risk label accuracy.** IP-related clauses are systematically under-classified due to a MEDIUM-biased distribution in the LLM-generated training labels. Attorney-verified labels would address this.
- 2) **Context window.** The 256-token Legal-BERT limit truncates long clauses, potentially losing critical context.
- 3) **Model size.** At 1.1B parameters, TinyLlama has limited reasoning depth compared to GPT-4-class models. Even with the Alternating Odd fine-tuning, complex multi-clause interactions may be missed.
- 4) **LLM-generated labels.** Risk training labels were produced by Llama 3.1 via the Groq API, not by human-verified attorneys.
- 5) **English only.** No multi-language support is currently implemented.
- 6) **Not legal advice.** The tool assists review and cannot replace qualified legal counsel.
- 7) **Fine-tuning scope.** The selective fine-tuning experiments focused on lower-layer configurations. Middle-layer combinations and longer training schedules remain unexplored.

X. CONCLUSION

LegallyBound demonstrates that effective, privacy-preserving contract risk analysis can be achieved with compact language models running on consumer-grade GPU hardware (NVIDIA RTX 4060, 8 GB VRAM). The system addresses all three identified industry gaps: the efficiency gap (automated pipeline with real-time streaming), the privacy gap (fully local deployment), and the accuracy gap (domain-adapted models).

The dual-model architecture — TinyLlama for clause extraction and Legal-BERT for risk classification — provides complementary strengths: generative capability for summarisation and discriminative precision for risk categorisation.

The fine-tuning research contributes the following finding: for highly divergent textual domains such as law, adapting the model’s fundamental lexical-processing capabilities through a **skip-pattern over lower transformer blocks** (Alternating Odd: layers 1, 3, 5, 7, 9) yields superior semantic understanding compared to standard top-block fine-tuning. The frozen layers between trained layers act as structural anchors that preserve each specialisation and eliminate gradient interference, achieving a +2.06 BERTScore F1 improvement over the zero-shot baseline.

Key findings are summarised:

- 1) The **Alternating Odd** selective fine-tuning configuration achieves BERTScore F1 of **84.32%** (+2.06 over zero-shot), outperforming all other layer combinations including the heavily parameterised Last 10 Layers approach (84.12%), confirming the efficacy of frozen structural anchors.
- 2) **Legal-BERT** fine-tuned on Groq-generated CUAD risk labels correctly classifies 4 of 6 representative clauses in independent verification, confirming the viability of LLM-assisted label generation for domain-specific classification tasks.
- 3) **Smart relevance filtering** at a cosine-similarity threshold of 0.25 eliminates approximately 38% of boilerplate chunks, directly reducing TinyLlama and Legal-BERT inference calls and improving end-to-end throughput.
- 4) **SSE streaming** substantially improves perceived responsiveness for long-running analyses by surfacing risk cards incrementally.
- 5) **Merged prompting** reduces per-chunk inference overhead from three to two model calls without degrading output quality.

A. Future Work

Planned extensions include: (1) attorney-verified risk label curation to address IP clause under-classification; (2) multi-lingual contract support; (3) continuous risk scoring with confidence intervals replacing the three-tier taxonomy; (4) side-by-side clause comparison across contracts; (5) integration with legal databases linking identified clauses to relevant case law; (6) evaluation of larger base models (Phi-2 2.7B; Mistral-7B) when hardware permits; (7) extension of the selective fine-tuning search to middle-layer configurations and LoRA hybrid strategies.

REFERENCES

- [1] D. Hendrycks, C. Burns, A. Chen, and S. Ball, “CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review,” *arXiv preprint arXiv:2103.06268*, 2021.
- [2] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, “LEGAL-BERT: The Muppets Straight out of Law School,” in *Findings of EMNLP 2020*, pp. 2903–2912, 2020.
- [3] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [4] P. Zhang *et al.*, “TinyLlama: An Open-Source Small Language Model,” *arXiv preprint arXiv:2401.02385*, 2024.
- [5] H. Touvron *et al.*, “LLaMA: Open and Efficient Foundation Language Models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [6] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating Text Generation with BERT,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [7] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30318–30332, 2022.
- [8] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of EMNLP-IJCNLP 2019*, pp. 3982–3992, 2019.
- [9] M. Raasveldt and H. Mühleisen, “DuckDB: an Embeddable Analytical Database,” in *Proceedings of SIGMOD 2019*, pp. 1981–1984, 2019.